

# Prisma Best Practices Guide

**Last Updated:** 2025-11-05 **Applies To:** Multi-Business Management Platform

## Table of Contents

- 1. [Overview](#)
- 2. [Naming Conventions](#)
- 3. [Safe Prisma Commands](#)
- 4. [Migration Workflows](#)
- 5. [Common Issues](#)
- 6. [Troubleshooting](#)

## Overview

This guide covers Prisma-specific best practices for the Multi-Business Management Platform. Following these conventions ensures consistency and prevents common deployment issues.

### Key Principles:

- Schema changes are made in development, then migrated to production
- Production **never** modifies the schema file
- All column names use camelCase
- All table names use snake\_case
- All model names use PascalCase

## Naming Conventions

### Required Conventions

Based on project standards from [CLAUDE.md](#):

```
// ✅ CORRECT Example:
model User {
  id          String    @id @default(cuid())
  email       String    @unique
  firstName   String    // camelCase column
  lastName    String    // camelCase column
  isActive    Boolean   @default(true)   // camelCase column
  createdAt   DateTime  @default(now())   // camelCase column
  updatedAt   DateTime  @updatedAt       // camelCase column

  businesses Business[]

  @@map("users") // snake_case table name
}
```

```

model Business {
  id          String   @id @default(cuid())
  businessName String
  businessType String
  shortName   String?  // camelCase column
  createdAt   DateTime @default(now())

  owner  User  @relation(fields: [ownerId], references: [id])
  ownerId String

  @@map("businesses") // snake_case table name
}

```

### Rules:

1. **Model names:** PascalCase (User, Business, BusinessCategory)
2. **Table names:** snake\_case via `@@map("table_name")`
3. **Column names:** camelCase (firstName, isActive, createdAt)
4. **Relation names:** descriptive camelCase (owner, businesses, categories)

### Why This Matters

#### Prisma generates client properties from model names:

```

// From model User { @@map("users") }
const users = await prisma.user.findMany() // ✅ Correct: camelCase client
property

// NOT this:
const users = await prisma.users.findMany() // ❌ Wrong

```

#### Seed scripts expect camelCase:

```

// ✅ CORRECT - matches generated Prisma client
await prisma.user.create({ ... })
await prisma.business.create({ ... })
await prisma.businessCategory.create({ ... })

// ❌ WRONG - will fail with "property not found"
await prisma.users.create({ ... })
await prisma.business_category.create({ ... })

```

## Safe Prisma Commands

- ✅ Production-Safe Commands

These commands are safe to run in production:

```
# Generate Prisma client from schema
npx prisma generate

# Apply pending migrations (production)
npx prisma migrate deploy

# Check migration status
npx prisma migrate status

# Open Prisma Studio (read-only recommended)
npx prisma studio
```

## ✗ Development-Only Commands

**NEVER run these in production** - they modify the schema file:

```
# ✗ DANGEROUS - Pulls schema from database, overwrites schema.prisma
npx prisma db pull

# ✗ DANGEROUS - Pushes schema without creating migration
npx prisma db push

# ✗ DANGEROUS - Creates new migration (development only)
npx prisma migrate dev

# ✗ DANGEROUS - Drops all data and resets database
npx prisma migrate reset
```

## Why These Are Dangerous

### **npx prisma db pull:**

- Overwrites your `schema.prisma` file
- Causes git conflicts
- May lose manual annotations and comments

### **npx prisma db push:**

- Bypasses migration history
- Can cause data loss
- No rollback capability

### **npx prisma migrate dev:**

- Creates new migration files
- Should only be done in development
- Requires manual commit to git

---

# Migration Workflows

## Development Workflow

When you need to change the database schema:

```
# 1. Modify schema.prisma in your editor
# Example: Add new field to User model

# 2. Create migration
npx prisma migrate dev --name add_user_phone_field

# 3. Prisma will:
#   - Generate SQL migration file
#   - Apply it to your dev database
#   - Regenerate Prisma client

# 4. Test your changes locally

# 5. Commit both schema.prisma AND migration files
git add prisma/schema.prisma
git add prisma/migrations/
git commit -m "Add phone field to users"
git push
```

## Production Deployment Workflow

When deploying schema changes to production:

```
# 1. Pull latest code (includes schema changes and migrations)
git pull origin main

# 2. Apply migrations
npx prisma migrate deploy

# 3. Regenerate Prisma client
npx prisma generate

# 4. Restart application
npm run service:restart
```

**Note:** The Windows service handles steps 2-4 automatically when you restart it.

## Checking Migration Status

```
# Check if migrations are pending
npx prisma migrate status
```

```
# Expected outputs:  
# "Database schema is up to date!" -  Good  
# "Following migrations have not yet been applied:" -  Run migrations
```

## Rolling Back Migrations

Prisma doesn't have built-in rollback. If you need to revert:

### Option 1: Create reverse migration (recommended)

```
# In development:  
# 1. Modify schema.prisma to revert change  
# 2. Create new migration  
npx prisma migrate dev --name revert_phone_field  
  
# 3. Deploy to production  
git push  
# (production will apply the revert migration)
```

### Option 2: Restore from backup (emergency)

```
# Stop service  
npm run service:stop  
  
# Restore database from backup  
psql -U postgres -d multi_business_db < backup.sql  
  
# Restart service  
npm run service:start
```

---

## Common Issues

Issue: Schema File Modified After Git Pull

### Symptoms:

```
git status  
# Shows: modified: prisma/schema.prisma
```

**Cause:** Someone ran `npx prisma db pull` in production

### Solution:

```
# Discard changes and restore from git
git checkout HEAD -- prisma/schema.prisma

# Verify schema is back to committed version
git status # Should be clean

# Apply proper migrations
npx prisma migrate deploy
```

**Prevention:**

1. Never run `prisma db pull` in production
2. Make schema changes in development only
3. Use migration workflow (above)

**Issue: Model Not Found on Prisma Client****Symptoms:**

```
TypeError: Cannot read properties of undefined (reading 'findMany')
```

**Cause:** Prisma client not regenerated after schema changes

**Solution:**

```
# Regenerate Prisma client
npx prisma generate

# Verify client generated
ls -la node_modules/.prisma/client/

# Restart application
npm run service:restart
```

**Issue: Migration Fails with "duplicate key" Error****Symptoms:**

```
Error: Migration failed to apply
Unique constraint violation
```

**Cause:** Data in database violates new constraint

**Solutions:**

## Option 1: Fix data first

```
# Connect to database
psql -U postgres -d multi_business_db

# Find duplicate data
SELECT "email", COUNT(*)
FROM users
GROUP BY "email"
HAVING COUNT(*) > 1;

# Fix duplicates manually
# Then retry migration
npx prisma migrate deploy
```

## Option 2: Modify migration

```
# Edit migration SQL file to handle duplicates
# Add data cleanup before constraint
nano prisma/migrations/XXXXXX_migration_name/migration.sql

# Example:
-- Remove duplicates first
DELETE FROM users a USING users b
WHERE a.id > b.id AND a.email = b.email;

-- Then add constraint
ALTER TABLE users ADD CONSTRAINT users_email_key UNIQUE (email);
```

## Issue: EPERM Error During Prisma Generate

### Symptoms:

```
Error: EPERM: operation not permitted
Could not copy Prisma Client files
```

**Cause:** Windows service has Prisma client files locked

### Solution:

```
# Stop service to release file locks
npm run service:stop

# Clear Prisma cache
rm -rf node_modules/.prisma/client
```

```
# Regenerate
npx prisma generate

# Restart service
npm run service:start
```

---

## Troubleshooting

### Diagnosing Schema Issues

```
# Check if schema is valid
npx prisma validate

# Check migration status
npx prisma migrate status

# Generate client and see what models are available
npx prisma generate --data-proxy

# Introspect database to see actual structure
# (Don't save the result!)
npx prisma db pull --print > /tmp/actual-schema.txt
diff prisma/schema.prisma /tmp/actual-schema.txt
```

### Migration Lock Issues

**Symptoms:** Migration hangs or times out

**Solution:**

```
# Check for lock file
ls -la prisma/.prisma-migration-lock

# If stuck, remove lock
rm prisma/.prisma-migration-lock

# Clear failed migrations in database
psql -U postgres -d multi_business_db

# Delete incomplete migration
DELETE FROM _prisma_migrations WHERE finished_at IS NULL;

# Retry migration
npx prisma migrate deploy
```

### Viewing Migration History

```
-- Connect to database
psql -U postgres -d multi_business_db

-- View all migrations
SELECT migration_name, finished_at, applied_steps_count
FROM _prisma_migrations
ORDER BY finished_at DESC;

-- Check for failed migrations
SELECT * FROM _prisma_migrations
WHERE finished_at IS NULL;
```

## Performance Issues

### Check query performance:

```
# Enable Prisma query logging in .env.local
DATABASE_URL="postgresql://...?logging=true"

# Or set in code:
const prisma = new PrismaClient({
  log: ['query', 'info', 'warn', 'error'],
})
```

### Add indexes for slow queries:

```
model Business {
  // ... fields ...

  @@index([ownerId]) // Add index for foreign key
  @@index([businessType, isActive]) // Composite index for filtering
}
```

---

## Quick Reference

### Development Commands

```
# Create migration
npx prisma migrate dev --name descriptive_name

# Reset database (WARNING: deletes all data)
npx prisma migrate reset

# Open database GUI
npx prisma studio
```

```
# Validate schema
npx prisma validate
```

## Production Commands

```
# Apply migrations
npx prisma migrate deploy

# Generate client
npx prisma generate

# Check status
npx prisma migrate status
```

## Emergency Recovery

```
# Restore schema file
git checkout HEAD -- prisma/schema.prisma

# Clear locks
rm prisma/.prisma-migration-lock

# Clean client cache
rm -rf node_modules/.prisma/client

# Regenerate everything
npx prisma generate
npm run build:service
```

---

## Additional Resources

- **Main Deployment Guide:** See [ADMIN-INSTALLATION-GUIDE.md](#) for full deployment workflow
  - **Prisma Documentation:** <https://www.prisma.io/docs>
  - **Migration Reference:** <https://www.prisma.io/docs/concepts/components/prisma-migrate>
  - **Schema Reference:** <https://www.prisma.io/docs/reference/api-reference/prisma-schema-reference>
- 

## Best Practices Summary

1.  **DO** create migrations in development and commit them
2.  **DO** use `prisma migrate deploy` in production
3.  **DO** follow naming conventions (PascalCase models, camelCase columns, snake\_case tables)
4.  **DO** test migrations on a copy of production data before deploying
5.  **DO** backup database before applying migrations

6. ❌ **DON'T** run `prisma db pull` in production
  7. ❌ **DON'T** run `prisma db push` in production
  8. ❌ **DON'T** modify `schema.prisma` in production
  9. ❌ **DON'T** run `prisma migrate dev` in production
  10. ❌ **DON'T** run `prisma migrate reset` in production
- 

**Document Version:** 1.0 **Last Updated:** 2025-11-05 **Project:** Multi-Business Management Platform